

Bird Species Observation Analysis

Project Blueprint & Execution Pipeline — v2

Owner	Tanay Nagpal — Labmentrix Internship
Timeline	7 days from assignment date
Stack	Python / pandas · Plotly · Streamlit · Git + GitHub · VS Code
Deployment	GitHub repository to Streamlit Community Cloud
Theme	"Dawn Chorus" — locked. Full palette in Section 4.
Status	Planning complete. 13 data decisions locked, theme locked, 10 tabs specified. Ready to build.
How to use	This is the controlling document. Work top to bottom. Section 3 (guardrails) is not optional — it is the difference between a correct project and a confidently wrong one.

1. What We Are Delivering

#	Deliverable	Format
1	Cleaned dataset with documented cleaning steps	CSV
2	Source code for cleaning, analysis and visualisation	Python, commented
3	Interactive dashboard, 10 tabs, deployed publicly	Streamlit
4	Written report of approach, findings and recommendations	PDF + in-app tab

Findings the pipeline must reproduce

- **Headline:** forest records at-risk species at ~6× the grassland rate (3.65% vs 0.59%), consistent across all four shared parks.
- **Null result:** once effort and park are controlled, species richness does not differ between habitats. The apparent grassland advantage is a sampling artefact.
- **Specialists:** 18 grassland specialists versus 1 forest specialist among 49 well-sampled species.
- **Hotspots:** ANTI, CHOH, NACE, WOTR lead on species per session.
- **Environment:** richness falls as temperature rises; drizzle and serious disturbance suppress counts.
- **Daily rhythm:** grassland richness declines through the morning; forest stays flat.

2. Repository Structure

```
bird-species-observation-analysis/  
  data/raw/          # the two source .xlsx files, never edited  
  data/processed/    # birds_clean.csv, birds_features.csv, sessions.csv,  
species_profile.csv  
  data/reference/     # park_coordinates.csv  
  src/ingest.py       # read all sheets from both workbooks  
  src/clean.py        # the 13 locked decisions  
  src/features.py     # the 24 calculated fields (Section 7)  
  src/monitoring.py   # period-agnostic monitoring layer  
  src/analysis.py     # guarded comparison functions - the 12 business questions  
notebooks/01_exploration.ipynb · 02_analysis.ipynb  
app/streamlit_app.py · charts.py · theme.py · qa_bank.py  
docs/cleaning_log.md · final_report.pdf  
requirements.txt · README.md
```

3. Analytical Guardrails

Four rules. Each one, if broken, produces a result that looks statistically valid and is wrong. All four were derived from actual tests on this dataset.

Rule	Evidence it is needed
G1. Never compare raw counts across habitats. Use per-session rates.	Grassland has 594 sessions to forest's 814. Raw totals measure survey effort, not birdlife.
G2. Every habitat comparison is made within a park, using only ANTI, HAFE, MANA, MONO.	Pooled across 11 parks: grassland richer, 9.15 vs 8.61, $p=0.0005$. Restricted to shared parks: 9.15 vs 9.10, $p=0.685$ — the effect vanishes. Two parks reverse direction.
G3. Use per-session, never per-plot.	Forest plots average 2.0 visits, grassland 2.96. Per-plot metrics inflate grassland by ~50%.
G4. Maps and charts encode adjusted rates, never raw totals.	Park ranking by raw species: MONO, MANA, ANTI, CHOH. By species per session: ANTI, CHOH, NACE, WOTR. A different answer.

Test before publishing any comparison: would this number change if one park had been surveyed twice as often? If yes, the metric is wrong.

4. Visual Theme — Nature / Illustrated

A cream page with a dark green illustrated sidebar. The feel is a field guide rather than a business dashboard: light and open, with warmth and life coming from a hand-drawn illustration layer rather than from the data colours. This theme supersedes the earlier dark variant.

	Hex	Role	Notes
	#12805c	Forest series	Deep forest green
	#b07d0a	Grassland series	Dark gold
	#c0392b	At-risk status	Reserved. Never a series colour. Always paired with a label.
	#f6f4ec	Page background	Cream
	#ffffff	Card surface	
	#e4e1d5	Card border	
	#e8e5da	Gridline	Recessive hairline
	#1a2620	Primary ink	
	#5c6a61	Secondary ink	
	#8a968d	Muted ink	Axis and labels
	#16311f	Sidebar gradient top	Fades to #1d4029
	#1b3a25	KPI card gradient top	Fades to #245030
	#2b6b45	Nav active	White text

Validation. Run against the cream surface #f6f4ec in light mode with all pairs: forest/grassland colour-vision-deficiency Delta E 8.7 (protan), normal-vision Delta E 18.4, both clear 3:1 contrast. The lighter green and gold steps (#1baf7a / #eda100) were tested and **rejected** — they fail the 3:1 contrast check on a light surface. Do not substitute colours without re-validating.

Illustration layer

- Hand-drawn inline SVG throughout — no raster images. A few kilobytes, scales to any screen, and recolours from a single variable.
- Sidebar foot: a layered forest scene — mist gradient, far pines, near pines, a stag silhouette, and five birds in flight.
- Logo: a stylised feather and tree mark. Navigation and KPI cards carry a matching 24-pixel icon set.
- Depth is built from three silhouette layers at different opacities, so the scene reads at any sidebar height.

One thing deliberately not copied from the reference design

The reference dashboard used four shades of green for four donut categories. Shades of a single hue are close to indistinguishable under colour-vision deficiency, so that is a genuine accessibility failure rather than a style choice. Green paired with gold keeps the nature feel while remaining readable. **Rule: never use shades of one hue for categorical data.**

Creative features — decided

Feature	Decision	Reasoning
Bird call audio in Species Explorer	BUILD	Not decoration: 63% of all records are birds identified by song, so hearing the call closes a real loop with the ID_Method column. Curated set of roughly 20 Creative Commons recordings from Xeno-canto, stored locally with attribution — not 127 live API calls.
UI click and hover sounds	SKIP	Irritating on repeat use, unreliable under browser autoplay policy, and reads as unserious on a graded analytical deliverable.
Ambient dawn chorus	OPTIONAL	Only with a clearly labelled toggle that is off by default.
Animated KPI counters	POLISH	Numbers tick up on load. Cheap and common in good dashboards.
Species silhouettes	POLISH	Same SVG style as the sidebar, shown in species profiles.
Time-of-day header tint	POLISH	Header gradient shifts between the 5am and 10am filters, tying the visual to the data.

Polish items are built only if Phase 7 finishes ahead of schedule. A working dashboard with one memorable feature beats a half-finished one with five.

5. Navigation Architecture

Ten tabs. Named for findings, not data types, so the navigation itself tells the story. Each tab opens with a one-sentence plain-language finding before any chart.

#	Tab	Heading shown	Objective
1	Overview	"Forest is the stronghold for at-risk birds"	1, 5
2	Habitat Comparison	"Comparing habitats fairly"	1, 5, 6
3	Species Explorer	"Which birds prefer which habitat"	2
4	Where to Watch	"Biodiversity hotspots across 11 parks"	3
5	Seasonal & Daily	"When birds are most active"	6
6	Environment	"How weather shapes what we see"	6
7	Data Quality	"How this analysis was built"	method
8	Report	"The full story, start to finish"	deliverable 4
9	Conclusion	"What we found and what to do"	all
10	Ask AI	"Ask anything about this data"	all

Persistent elements

- **Filter bar** (one row, above content, all tabs): habitat · park · date range · species search · at-risk-only toggle.
- **Sidebar**: project title, dataset facts, navigation, export buttons, method link, GitHub link, theme toggle.
- **Guardrail banner** on Overview and Habitat Comparison: "All comparisons shown per survey session and within park."
- **Limitations panel** (collapsible, Overview): single season; no land-use data; grassland covers 4 of 11 parks.
- **Downloads**: cleaned CSV, current filtered view, and the report PDF.

6. Chart Specifications

Seventeen charts. Form is chosen by the data's job, not by variety. No dual-axis charts anywhere; a legend is present whenever two series appear; every chart carries a tooltip and a one-line interpretation beneath it.

#	Tab	Chart	Form	Why this form
1	Overview	Headline stat tiles	Stat tiles	Single values — not a chart
2	Overview	"6x" at-risk hero	Hero number	One number is the whole story
3	Overview	Most recorded species	Stacked h-bar	Magnitude + habitat split in one
4	Habitat	At-risk rate by park	Grouped bar	Magnitude across two categories
5	Habitat	Richness per session by park	Grouped bar	Same form — comparison is the point
6	Habitat	Survey effort by park	Grouped bar	Context for the two above
7	Species	Habitat preference	Diverging bar	Data has genuine polarity
8	Species	Specialist breakdown	Bar	Simple category magnitude
9	Species	Per-species profile	Card + sparkline	Detail view, not comparison
10	Where	Park map	Bubble map	Spatial — adjusted rates only (G4)
11	Where	Parks ranked	Horizontal bar	Ranking with long labels
12	Where	Park × habitat coverage	Heatmap	Two-dimensional density
13	Seasonal	Richness by month	Line	Change over time
14	Seasonal	Richness by hour	Line	The clean temporal story
15	Environment	Temperature vs richness	Binned bar	1,408 raw points would be mush
16	Environment	Sky / wind / disturbance	Small multiples	Three parallel comparisons
17	Data Quality	Observer comparison	Bar	Bias disclosure

7. Calculated Fields

Twenty-four derived fields, built in `src/features.py` after cleaning, producing four tables: row level, session level, species level and the park lookup.

Group	Field	Definition
Identity	session_id	Plot_Name + Date + Visit — the effort unit. Everything depends on this.
	habitat	From Location_Type
	park_name	Full park name from the 4-letter code
	is_shared_park	TRUE for ANTI / HAFE / MANA / MONO. Guardrail G2 filters on this.
Time	month, month_name	From Date
	week	Computed but never charted (decision #12)
	year, quarter	For the auto-scaling monitoring layer
	start_hour	Hour from Start_Time
	time_band	Early 5–6am / Mid 7–8am / Late 9–10am
	session_duration_min	End_Time – Start_Time
	day_of_season	Days since 7 May 2018
Cleaned	sex_clean	Blanks to "Undetermined" (decision #6)
	is_at_risk	PIF watchlist flag as boolean
	is_stewardship	Regional stewardship flag as boolean
	is_flyover	Flyover_Observed as boolean
	distance_display	Blank becomes "Flyover (n/a)" so flyovers do not vanish from distance charts (decision #7)
Metrics	species_per_session	Distinct species per session — the primary metric
	sightings_per_session	Row count per session
	pct_at_risk_per_session	Share of sightings that are watchlist species
Species	grassland_share_pct	Percent of a species' sightings in grassland
lookup	specialist_class	Grassland specialist / Forest specialist / Generalist
	total_sightings	Used to filter to well-sampled species (at least 20)
Binning	temp_band, humidity_band	For binned environmental charts
Reference	lat, lon	Per park — 11 rows from public NPS sources, cited

8. Environment Constraints and Coding Standards

Every item below was found by auditing the actual development machine. Each one produces a failure that is silent locally and only appears at deployment, which is the worst kind. They are not style preferences.

Environment as built

Item	Value	Note
Python	3.13.15 (python.org)	NOT the Microsoft Store build - that one hangs on venv creation. Store aliases for python.exe / python3.exe / python3.13.exe are disabled.
Launcher	py	`python` is not on PATH outside the venv. Use `py -m venv .venv` to create; inside an activated venv `python` works normally.
Project path	D:\projects\bird-analysis	Deliberately short. The original path was 108 characters before any files, close to the 260-character Windows limit that breaks pip installs.
pandas	3.0.5	Major version 3, not 2. Older tutorials and Stack Overflow answers may use idioms that no longer apply.
scipy	REMOVED	Compiled DLLs blocked by Windows Smart App Control. Replaced by src/stats_helpers.py - see below.

The scipy replacement

scipy is not installed and must not be added. Its compiled extensions are blocked by Smart App Control on this machine, and the same class of restriction can appear on deployment hosts. The two tests the project needs - Mann-Whitney U and Spearman rank correlation - are implemented in **src/stats_helpers.py** using numpy and the standard library only, including an exact Student t distribution via the incomplete beta function. Both were validated against scipy on the real project data and match to machine precision: Mann-Whitney U=215458.5 p=0.000450024 and the four-park null result U=41549.5 p=0.685483 are identical to scipy's output, and Spearman rho and p agree to 1e-12 across every tested pair. Removing scipy leaves the project more robust, not less - one fewer dependency that can fail on deploy, and the statistical method is visible in our own code rather than hidden in a library.

Coding standards - mandatory

Rule	Why
E1. Every file write specifies <code>encoding="utf-8"</code> . This includes <code>open()</code> , <code>to_csv()</code> , <code>read_csv()</code> and every reportlab and markdown write.	Python on Windows defaults to cp1252, not UTF-8. The bird data is pure ASCII (verified - zero non-ASCII characters in either workbook), so the cleaned CSV survives, but the cleaning log and report contain dashes and arrows that will corrupt or raise UnicodeEncodeError.
E2. Never use PowerShell <code>></code> to create a file a program will read. Use <code> Out-File -Encoding ascii</code> .	PowerShell redirection writes UTF-16. pip cannot read a UTF-16 requirements.txt, and the failure only surfaces at deployment as a confusing dependency error.
E3. All paths built with pathlib, never string concatenation. All filenames lowercase.	Windows is case-insensitive; the Linux deploy host is not. One capital letter difference works locally and fails in production with "file not found".
E4. Two dependency files. requirements.txt is runtime only (6 packages). requirements-dev.txt adds Jupyter.	A full pip freeze produced 119 packages including 22 Jupyter ones the app never uses. Each is another chance for the deploy to fail, and the install is far slower.
E5. Pin the Python version for the deploy host in runtime.txt.	Without it the host picks its own default, which may not match the version the code was tested against.
E6. Commit .gitattributes with eol=lf and mark xlsx, db, png and pdf as binary.	Prevents CRLF/LF churn in diffs and stops git corrupting binary files through line-ending conversion.

Rule	Why
E7. Test that compiled packages actually run, not just import.	scipy imported successfully at the top level and only failed when loading a compiled submodule. pyarrow and pillow are also compiled and are Streamlit dependencies - exercise a real function on each.

Data note: the Wind column contains 138 values with an embedded comma, for example "Gentle breeze (8-12 mph), leaves in motion". pandas quotes these correctly on CSV export and re-reads them correctly, but any downstream consumer that splits on commas naively will corrupt those rows.

9. Execution Pipeline

Phase 0 — Environment setup

Step	Task	Detail / acceptance criteria
0.1	Create GitHub repository	Public, named bird-species-observation-analysis. Python .gitignore.
0.2	Clone and open in VS Code	Confirm interpreter selected.
0.3	Virtual environment	py -m venv .venv then .venv\Scripts\Activate.ps1. Use `py`, not `python` - see Section 8.
0.4	Install dependencies	pip install -r requirements-dev.txt. NOT a raw pip freeze - see Section 8 rule E4. scipy is deliberately excluded.
0.5	Create folder structure	Exactly as Section 2. Commit the skeleton.
0.6	Copy raw files to data/raw/	Lowercase .xlsx extension. Never edit these.
0.7	Add config files	requirements.txt, requirements-dev.txt, runtime.txt, .gitattributes, .gitignore per Section 8.
0.8	Verify compiled packages run	Exercise a real function in pyarrow and pillow, not just import. Rule E7.

Phase 1 — Ingestion

Step	Task	Detail / acceptance criteria
1.1	Read all sheets	pd.read_excel(path, sheet_name=None) — 11 sheets per file.
1.2	Concatenate within each file	Expect 8,546 forest and 8,531 grassland rows. Seven empty grassland sheets are expected.
1.3	Verify habitat tag	Location_Type must hold one value per file.
1.4	Assert row counts	Fail loudly if counts differ from the above.

Phase 2 — Cleaning (the 13 locked decisions)

Order matters. Reconcile schema before combining; deduplicate before any counting.

Step	Task	Detail / acceptance criteria
2.1	Rename species code	NPSTaxonCode and TaxonCode both become TaxonCode. [#2]
2.2	Drop Previously_Obs	Constant FALSE across all grassland rows. [#3]
2.3	Keep Site_Name nullable	Real for forest, blank for grassland. [#1]
2.4	Keep Sub_Unit_Code	Blank means the park has no sub-units — that pattern is information. [#5]
2.5	Cast grassland types	Ints, floats, booleans, datetime, time. [#9]
2.6	Combine the two files	pd.concat — append rows, not merge. Expect 17,077.
2.7	Remove duplicates	Grassland only: 8,531 to 6,826, removing 1,705. Log before and after. [#4]
2.8	Standardise Sex	Forest blanks to "Undetermined". Never infer sex. [#6]
2.9	Leave Distance nulls	Null exactly when flyover. Structural. [#7]
2.10	Leave small gaps	Scientific_Name is the species key. [#8]
2.11	Write cleaning_log.md	Every decision with row counts and reasoning. Graded deliverable.
2.12	Export cleaned data	birds_clean.csv — expect 15,372 rows.

Phase 3 — Calculated fields

Step	Task	Detail / acceptance criteria
3.1	Build all 24 fields	Per Section 7, in src/features.py.
3.2	Build park_coordinates.csv	11 rows from public NPS sources. Cite the source in the file header.
3.3	Verify session count	Distinct session_id must equal 1,408.

Phase 4 — Exploratory data analysis

Step	Task	Detail / acceptance criteria
4.1	Distributions	Species frequency, sightings per park, habitat, plot.
4.2	Temporal	Monthly only — never weekly. Time-of-day is primary. [#12]
4.3	Environmental	Temperature and humidity against richness. Expect temperature rho ~ -0.2, highly significant.
4.4	Conditions	Expect drizzle ~7.0 and serious disturbance ~5.7 species per session against ~9.2 in good conditions.
4.5	Observer	Report spread and balance checks. Do not correct. [#13]
4.6	Detection method	Forest 5% visual vs grassland 20% — a caveat on habitat comparisons.

Phase 5 — Guarded analysis — the 12 business questions

Step	Task	Detail / acceptance criteria
5.1	Richness by habitat	Within-park, per session. Expect no significant difference. Report the null result.
5.2	At-risk by habitat	Within-park, per session. Expect forest ~3.65% vs grassland ~0.59% in all four parks. The headline.
5.3	Robustness	Re-run 6.2 separately per observer. All three must agree in direction.
5.4	Specialists	Species with at least 20 sightings in shared parks. Expect 18 grassland, 1 forest, 30 generalist.
5.5	Hotspots	Parks and plots by species per session. Note how this differs from raw ranking.
5.6	Sex ratio	Grassland only — forest has zero Female records. [#6]
5.7	Monitoring layer	Monthly now; auto-detects quarterly and yearly if more years arrive. Flag partial periods.

Phase 6 — Dashboard (tabs 1–7)

Step	Task	Detail / acceptance criteria
6.1	Theme and art modules	app/theme.py holds the Section 4 palette; app/art.py holds the SVG illustration set. All charts and chrome read from these two files only.
6.2	App skeleton	Tabbed layout reading the processed CSVs, cached with st.cache_data.
6.3	Filter bar and sidebar	Per Section 5.
6.4	Tabs 1–7	Charts 1–17 per Section 6, each with its interpretation line.
6.5	Guardrail banner	On Overview and Habitat Comparison.
6.6	Map caveat	Visible note: circles show survey locations and adjusted rates, not bird density.
6.7	Bird call audio	Curate ~20 Creative Commons recordings from Xeno-canto for the most-recorded species. Store in app/audio/ with an attribution file. Play button on each species profile.
6.8	Polish (if time allows)	Animated KPI counters, species silhouettes, time-of-day header tint. Skip without regret if behind schedule.

Phase 7 — Report, Conclusion and Ask AI (tabs 8–10)

Tab 8 — Report. A long-form single-column scroll, not a chart grid: executive summary, dataset, cleaning, method and guardrails, findings, recommendations, limitations. The PDF is **pre-generated and committed**, not built live — the report tells one fixed story, and live generation is slow and crash-prone on the free tier. A separate "export current view as CSV" button covers the interactive case.

Tab 9 — Conclusion. The decision-maker's page, distinct from the Report. Each item structured as Finding then What it means then Recommended action then Confidence, grouped by business objective. The confidence column is not decoration: the at-risk finding survived every robustness check, the specialist finding is solid, the environmental correlations are real but modest, and the seasonal signal is weak and confounded. This tab also carries the honest items — the null result, the unanswerable Sustainable Agriculture objective, and the single-season limitation.

Tab 10 — Ask AI. Rule-based intent matching where every answer is **computed live** from the processed tables. No API key, no cost, works permanently on the free tier, and it cannot hallucinate a number. Describe it accurately as a natural-language query interface over the dataset.

Step	Task	Detail / acceptance criteria
7.1	Report tab layout	Single column, narrative text, charts embedded where they support a point.
7.2	Generate report PDF	reportlab. Commit to docs/final_report.pdf. Serve via st.download_button.
7.3	Conclusion tab	Finding / meaning / action / confidence, grouped by objective.
7.4	Build qa_bank.py	Seven intent categories per Section 11. Each intent maps to a real computation over the processed tables.
7.5	Guardrail-aware answers	Questions like "which habitat has more birds" must return the effort-adjusted answer, never the raw count. See Section 10.
7.6	Chat UI	st.chat_message and st.chat_input. Show suggested questions as clickable chips.
7.7	Fallback	Unmatched questions return a helpful "here is what I can answer" list, never a guess.

Phase 8 — Deployment

Step	Task	Detail / acceptance criteria
8.1	Commit everything	Including the processed CSVs and final_report.pdf so the deployed app has them.
8.2	Push to GitHub	requirements.txt complete and pinned.
8.3	Deploy to Streamlit Cloud	Entry point app/streamlit_app.py.
8.4	Test live	Every filter, tab, download and chat response. Check load time and mobile layout.
8.5	Add the live link	To README and the final report.

10. The Twelve Business Questions

Originally planned as an SQL file. SQL was removed from the project on 28 August 2026 - see the note below - so these are implemented instead as documented functions in `src/analysis.py`, one per question, each enforcing the guardrails internally. The questions, the guardrails and the documentation requirement are unchanged; only the language is.

#	Question	Objective
Q1	At-risk species rate by habitat, within each shared park	1, 5 — headline
Q2	Species richness per session by habitat, within each shared park	6 — null result
Q3	Top parks and plots by species per session	3 — hotspots
Q4	Habitat preference per species, shared parks only	2 — specialists
Q5	Which watchlist species occur where	1, 5
Q6	Monthly species richness by habitat	6
Q7	Richness by time-of-day band	6
Q8	Richness against temperature and humidity bands	6
Q9	Richness by sky, wind and disturbance category	6
Q10	Observer session counts and mean richness	method
Q11	Species recorded in one habitat only	2
Q12	Survey effort by park and habitat	method

Note: why SQL was removed

The brief lists SQL twice - in the skills table, and in the guidelines line "Store data in an SQL database for visualization." The project owner decided on 28 August 2026 to drop it and run the pipeline as pandas straight into Streamlit, consistent with the approach used on earlier projects.

This is a known, deliberate departure from the brief and should be stated as such in the final report rather than left for a reviewer to notice. The analytical substance is unaffected: the same twelve questions are answered, the same four guardrails are enforced, and the answers are documented in code. What is lost is the demonstration of SQL as a skill and the literal compliance with that guideline.

Everything the SQL layer would have provided is preserved elsewhere: the session table in Phase 3 already does the work a normalised schema would have done, and `analysis.py` provides the documented, reusable query layer.

11. Ask AI — Question Bank

Category	Example questions	Answer source
Personal	hi · hello · who are you · what can you do · thanks · bye · help	Static, friendly
Dataset facts	how many species · how many records · which parks · what time period · who collected this	Computed count
Headline findings	which habitat is better for at-risk birds · what is the main conclusion · is forest or grassland richer	Computed + guardrail text
Species lookup	tell me about [species] · is [species] at risk · where do I find [species] · most common bird	Live query
Park lookup	tell me about [park] · best park for birdwatching · which park has most species	Live query

Category	Example questions	Answer source
Methodology	why remove duplicates · why per session · why only four parks · what are the limitations	Static, from cleaning log
Guardrail-protected	which habitat has more birds · which park has the most sightings	Adjusted answer — see below

The guardrail-protected category is the one that matters. Asked "which habitat has more birds?", a naive bot answers "grassland, 8,531 sightings" — precisely the false conclusion the entire guardrail system exists to prevent. The required answer is: *"Raw counts favour grassland, but that reflects survey effort, not bird abundance. Adjusted per survey session and compared within parks, the habitats show no significant difference in richness — though forest holds far more at-risk species."* Built this way the chatbot defends the analysis instead of being a hole in it.

12. Seven-Day Schedule

Day	Focus	Completion criteria
1	Setup, ingestion, cleaning	birds_clean.csv exists with 15,372 rows; cleaning_log.md written.
2	Calculated fields	All 24 fields built across four tables; session count verified at 1,408.
3	EDA	Exploration notebook complete; environmental and observer patterns confirmed.
4	Guarded analysis (12 questions)	Headline, null result, specialists and hotspots reproduced with guardrails applied.
5	Dashboard tabs 1–7	All charts functional locally with filters working, in the Nature theme. Bird call audio wired into Species Explorer.
6	Tabs 8–10 and report	Report tab, PDF generated and committed, Conclusion written, Ask AI answering all seven categories.
7	Deploy and verify	Live on Streamlit Cloud; all links and downloads working; final review against the rubric.

13. Risks and Mitigations

Risk	Mitigation
Publishing a pooled habitat comparison by accident	Guardrail G2. Every comparison function takes a park parameter and defaults to shared parks only.
A chart sized by raw counts	Guardrail G4. All chart inputs come from rate columns.
The chatbot contradicts the dashboard	Rule-based only, every answer computed from the same CSVs the charts read. No generative model.
Streamlit Cloud cannot find the data files	Commit the processed CSVs and use pathlib relative paths throughout (rule E3).
Running out of time	Build Overview and Habitat Comparison first — they carry the headline. Tabs 8–10 are additive.
Reviewer questions the duplicate removal	cleaning_log.md records row counts, the 135-times-repeated example, and manager approval.
Reviewer notes the missing SQL requirement	State the departure openly in the report rather than hoping it is missed. See section 10.

Final note

The analysis is only as trustworthy as the guardrails. Two of the most natural things to do with this data — pooling all eleven parks, and comparing raw sighting counts — each produce a statistically significant result that is false. The pipeline above, the guarded analysis functions, and the rule-based chatbot are all designed so those mistakes are difficult to make by accident. Follow it in order and the headline finding will hold up to scrutiny.